Design and Development of a Multi-Agent Intelligent System for Automated Customer Complaint Classification

Dataset: https://www.kaggle.com/datasets/anandshaw2001/consumer-complaints

Student Name:

Student ID:

Table of Contents

1. Application and Workflow Selection 2

1.1 Introduction and Motivation 4

1.2 Explanation of the Selected Workflow with Diagram 4

1.3 Objectives of the Selected Intelligent System.....7

1.4 Impact of the Selected Project if Developed.....7

2. Selection of AI Technologies 7

2.1 Identification of Similar Systems 7

2.2 Innovative and Original Approaches 8

2.3 Justification for Chosen AI Methods and Technologies 9

3. Simulations and Development Setup 10

3.1 Simulation Environments and Setup Parameters 10

3.2 Agent-Wise Task Distribution, Functionality, and Datasets 11

3.3 Encoding and Data Processing Prior to Simulation 14

4. Use Case Explanation with Evidence.....14

4.1 Step-By-Step Workflow Execution 15

4.2 Developed Interface and External Interaction 17

5. Conclusions.....17

Reference.....18

List of Figures:

Figure 1 Workflow Diagram.....6

Figure 2 Dataset Overview 8

Figure 3 Agent 1 : Data load and feature engineering 8

Figure 4Agent 2:Training Preprocessing ,SMOTE and Encoding 9

Figure 5Jupyter-based workflows.....10

Figure 6Work Setup 11
Figure 7 Agent 4 Model training and output 13
Figure 8Encoding and Data Processing 14
Figure 9full classification reports 16
Figure 10 Feature selection 17

1. Application and Workflow Selection

1.1 Introduction and Motivation

Automated customer complaint management has emerged as one of the most urgent needs of the modern organizations that are service-based. As the number of interactions between organizations and their customers grows fast through digital systems, organizations are sometimes unable to control, categorize, and address complaints in real time. The way complaints are processed manually is tedious, inaccurate, and consumes a lot of resources. Late detection of the categories of complaints may cause losses to customer confidence, quality of service, and a brand name(Haj Qasem et al., 2023). The intelligent automation has thus turned out to be a strategic need rather than an optional upgrade.

In this project, the emphasis is on the creation of the multi-agent intelligent system, which would be able to automatically analyze the customer complaint data and categorize the issues into the particular categories. With machine learning and agentic AI concepts, each agent puts in a specialized service like data preparation, preprocessing, training, and evaluation of the model. Workflow is more modular, scalable, and explainable with the integration of multiple agents. This architecture is similar to the way that modern customer-service departments work, with tasks being segregated into specialized units in this architecture(Yansari, Ajoudani and Mosayyebi, 2023). This is mainly driven by the need to show how a smart system can help organizations to deal with huge pools of complaint data in an efficient manner and minimize the number of human hands in handling the system and enhancing the accuracy of decisions.

1.2 Explanation of the Selected Workflow with Diagram

[redacted]

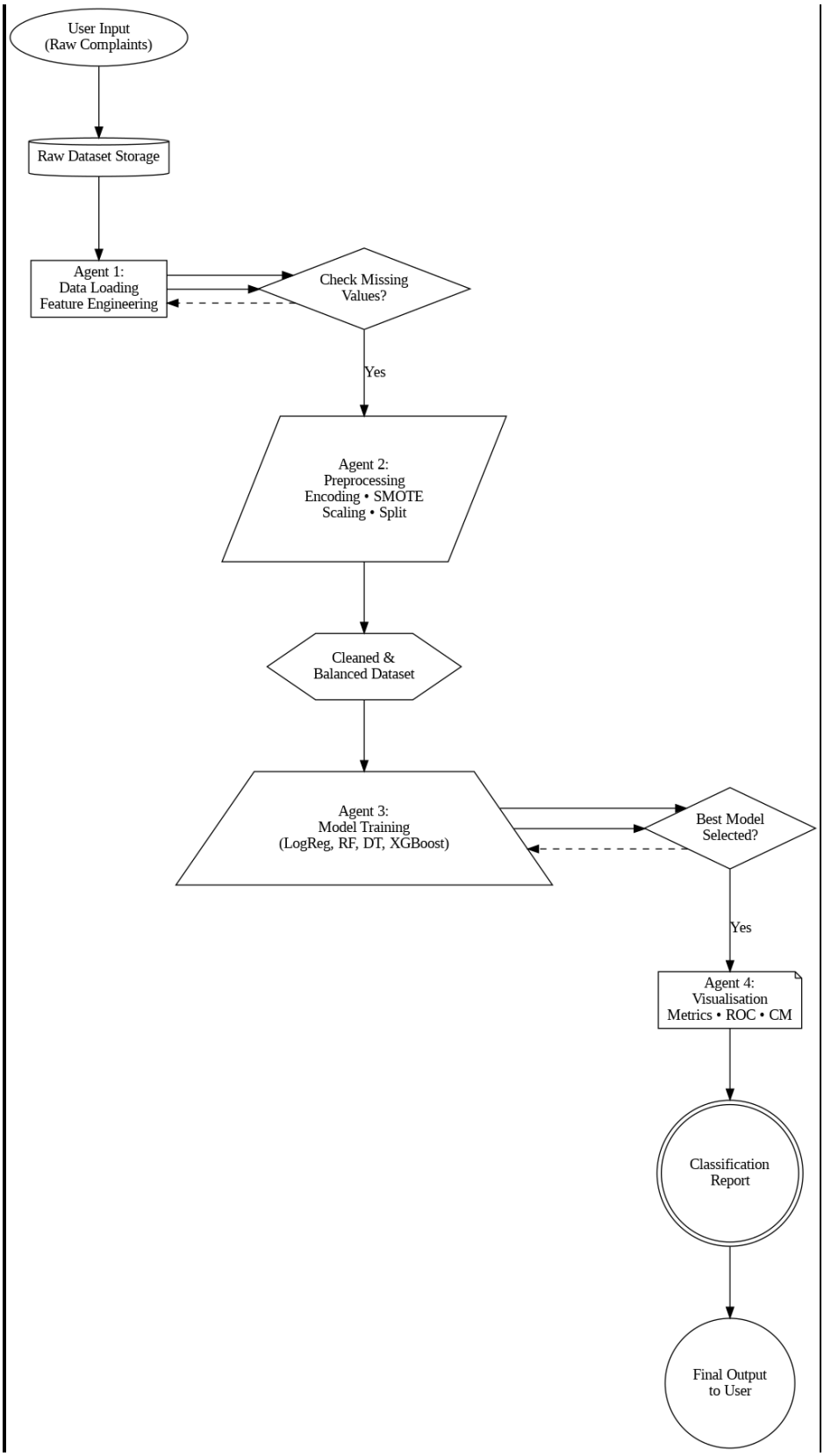
[redacted]

[redacted]

[redacted]

[redacted]

[redacted]


User Input
(Raw Complaints)
Raw Dataset Storage
Agent 1:
Data Loading
Feature Engineering
Check Missing
Values?
Yes
Agent 2:
Preprocessing
Encoding • SMOTE
Scaling • Split
Cleaned &
Balanced Dataset
Agent 3:
Model Training
(LogReg, RF, DT, XGBoost)
Best Model
Selected?
Yes
Agent 4:
Visualisation
Metrics • ROC • CM
Classification
Report
Final Output
to User

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```python
    # Output requested summaries
    print("Top 10 rows of dataframe:")
    display(df.head(10))
    print("\nDataset info:")
    print(df.info())
    print("\nNull values per column:")
    print(df.isnull().sum())
    print("\nColumn data types:")
    print(df.dtypes)
    print("Agent 1 completed")

    return df, feature_info

ZIP_PATH = "Consumer Complaints.zip"
df, feature_info = agent1(ZIP_PATH)
```

```
Agent 1 started
Top 10 rows of dataframe:
```

	ID	Company	Product	Issue	State	Submitted via	Date received	Date resolved	re
0	1615767	PHH Mortgage	Mortgage	Loan servicing, payments, escrow account	FL	Web	2015-10-20	10/20/2015	
1	654223	Ocwen	Mortgage	Loan servicing, payments, escrow account	NC	Web	2014-03-01	3/1/2014	
2	1143398	Southwest Credit Systems,	Debt collection	Loan modification,collection,foreclosure	MO	Web	2014-04-12	4/12/2014	

[redacted]

```
Dataset info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14000 entries, 0 to 13999
Data columns (total 21 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   ID                        14000 non-null  int64
 1   Company                   14000 non-null  object
 2   Product                   14000 non-null  object
 3   Issue                     14000 non-null  object
 4   State                     13890 non-null  object
 5   Submitted via             14000 non-null  object
 6   Date received             14000 non-null  datetime64[ns]
 7   Date resolved             14000 non-null  object
 8   Timely response?          14000 non-null  object
 9   Consumer disputed?        14000 non-null  object
 10  state name                13843 non-null  object
 11  Date received.1           14000 non-null  object
 12  Date resolved.1           14000 non-null  object
 13  Resolution time(in days)  14000 non-null  int64
 14  Year                      14000 non-null  int64
 15  QTR (US FLY)              14000 non-null  object
 16  _target_raw_              14000 non-null  object
 17  target                    14000 non-null  int32
 18  year                      14000 non-null  int32
 19  month                     14000 non-null  int32
 20  dayofweek                 14000 non-null  int32
dtypes: datetime64[ns](1), int32(4), int64(3), object(13)
memory usage: 2.0+ MB
```

```
Null values per column:
ID                          0
Company                     0
Product                     0
Issue                       0
State                     110
Submitted via               0
Date received               0
Date resolved               0
Timely response?            0
Consumer disputed?          0
state name                157
Date received.1             0
Date resolved.1             0
Resolution time(in days)    0
Year                        0
QTR (US FLY)                0
_target_raw_                0
target                      0
year                        0
month                       0
dayofweek                   0
dtype: int64

Column data types:
ID                                int64
Company                          object
Product                          object
Issue                            object
State                            object
```

[redacted]

[redacted]

```
Agent 2 completed
X_train_trans shape: (11200, 1162)
X_test_trans shape: (2800, 1162)
X_res (after SMOTE) shape: (17380, 1162)
y_res (after SMOTE) shape: (17380,)
First 5 rows of preprocessed training data:
 [[ 1.27510606  0.53595137  1.31476936 ...  0.          0.
   0.        ]
 [ 0.31939517  0.53595137 -0.33126427 ...  0.          0.
   0.        ]
 [-0.63631573  0.23275633 -1.42862002 ...  0.          0.
   0.        ]
 [-0.63631573  1.14234144 -0.33126427 ...  0.          0.
   0.        ]
 [-0.63631573  0.8391464  -1.42862002 ...  0.          0.
   0.        ]]
```

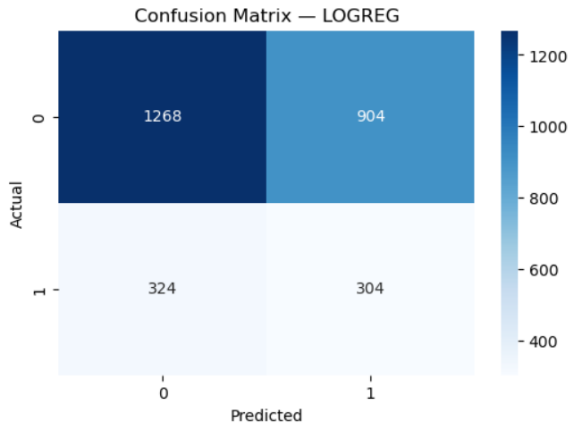
```python
def agent3(X_res, y_res, X_test_trans, y_test, random_state=42):
    models = {
        "logreg": LogisticRegression(max_iter=1000, random_state=random_state, class_weight="balanced"),
        "rf": RandomForestClassifier(random_state=random_state, n_jobs=-1),
        "dt": DecisionTreeClassifier(random_state=random_state),
        "xgb": xgb.XGBClassifier(use_label_encoder=False, eval_metric="logloss", random_state=random_state, n_jobs=-1)
    }

    model_results = {}
    best_f1 = -1
    best_model = None
    best_name = None

    for name, model in models.items():
        model.fit(X_res, y_res)
        try:
            y_prob = model.predict_proba(X_test_trans)[:,1]
            y_pred = (y_prob >= 0.5).astype(int)
        except Exception:
            y_pred = model.predict(X_test_trans)
            y_prob = None

        acc = accuracy_score(y_test, y_pred)
        prec = precision_score(y_test, y_pred, zero_division=0)
        rec = recall_score(y_test, y_pred, zero_division=0)
        f1 = f1_score(y_test, y_pred, zero_division=0)
        roc = roc_auc_score(y_test, y_prob) if y_prob is not None else None
```

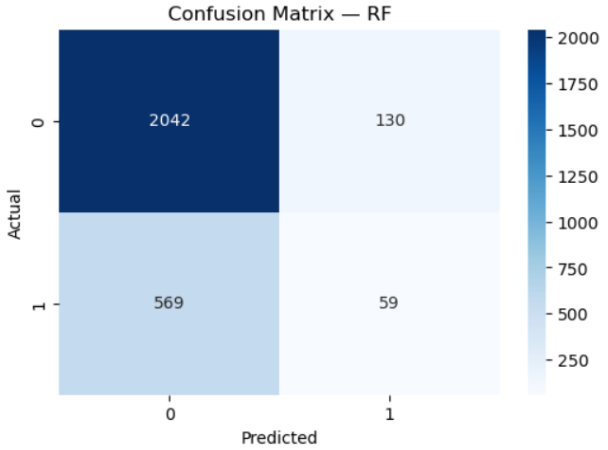
```python
import os
import zipfile
import warnings
import time
from typing import Tuple, Dict, Any, List

warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, RandomizedSearchCV, StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, classification_report, confusion_matrix, RocCurveDisplay
)
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier

import xgboost as xgb
```

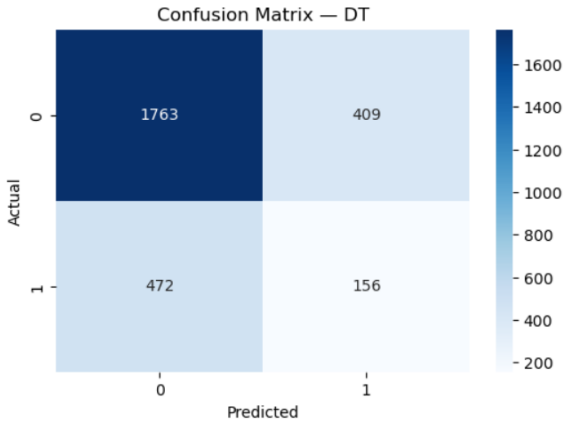
```
Agent 4 started

Model: logreg
Accuracy: 0.5614285714285714
              precision    recall  f1-score   support

           0       0.80      0.58      0.67      2172
           1       0.25      0.48      0.33       628

    accuracy                           0.56      2800
   macro avg       0.52      0.53      0.50      2800
weighted avg       0.67      0.56      0.60      2800


Model: rf
Accuracy: 0.7503571428571428
              precision    recall  f1-score   support

           0       0.78      0.94      0.85      2172
           1       0.31      0.09      0.14       628

    accuracy                           0.75      2800
   macro avg       0.55      0.52      0.50      2800
weighted avg       0.68      0.75      0.69      2800
```

```
Model: dt
Accuracy: 0.6853571428571429
              precision    recall  f1-score   support

           0       0.79      0.81      0.80      2172
           1       0.28      0.25      0.26       628

    accuracy                           0.69      2800
   macro avg       0.53      0.53      0.53      2800
weighted avg       0.67      0.69      0.68      2800


Model: xgb
Accuracy: 0.7671428571428571
              precision    recall  f1-score   support

           0       0.78      0.97      0.87      2172
           1       0.38      0.06      0.10       628

    accuracy                           0.77      2800
   macro avg       0.58      0.52      0.49      2800
weighted avg       0.69      0.77      0.70      2800
```

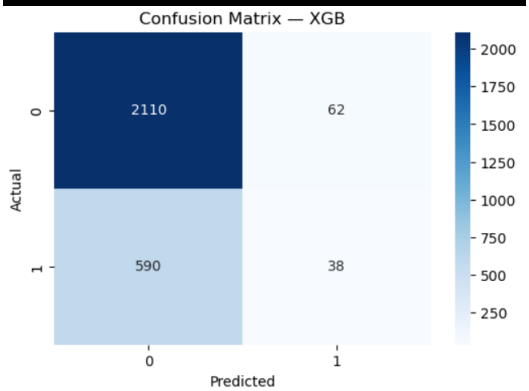
[redacted]

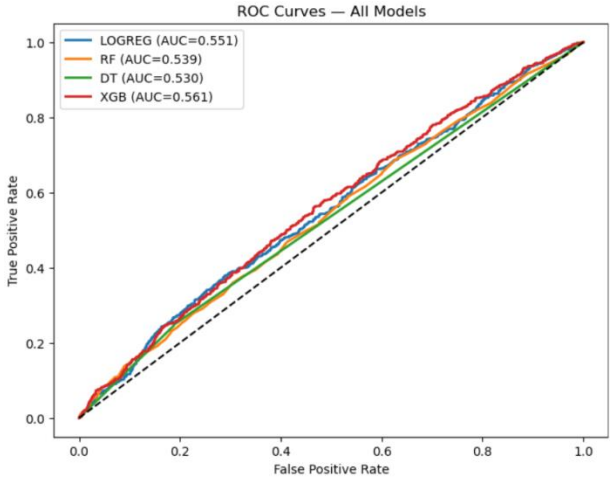






[redacted]





[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```python
    transformers.append(
        ("num", Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler())
        ]), feature_info["numeric_feats"])
    )

if feature_info["categorical_feats"]:
    transformers.append(
        ("cat", Pipeline([
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False))
        ]), feature_info["categorical_feats"])
    )

if feature_info["text_col"]:
    transformers.append(
        ("text", TfidfVectorizer(max_features=3000, stop_words="english"), feature_info["text_col"])
    )

preprocessor = ColumnTransformer(transformers, remainder="drop", verbose_feature_names_out=False)
preprocessor.fit(X_train_raw)

# Transform
X_train_trans = preprocessor.transform(X_train_raw)
X_test_trans = preprocessor.transform(X_test_raw)

if hasattr(X_train_trans, "toarray"):
    X_train_trans = X_train_trans.toarray()
    X_test_trans = X_test_trans.toarray()

# Apply SMOTE
if IMBLEARN_AVAILABLE:
    sm = SMOTE(random_state=random_state)
```

[redacted]

[redacted]

[redacted]

[redacted]

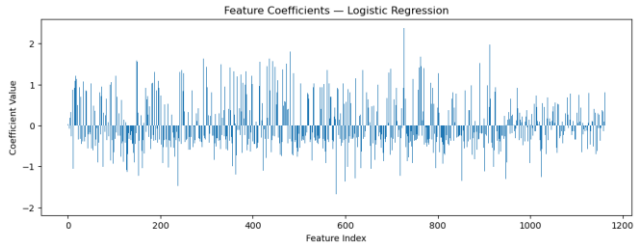
[redacted]

[redacted]

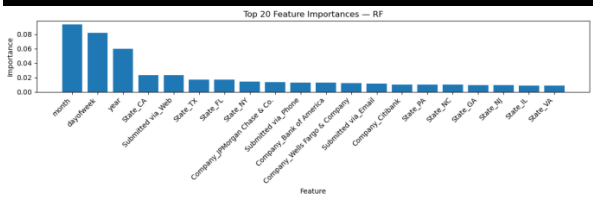
```
Agent 3 completed
Best model selected: logreg

logreg metrics:
Accuracy: 0.5614285714285714
Precision: 0.25165562913907286
Recall: 0.4840764331210191
F1 score: 0.3311546840958606
ROC AUC: 0.5507277773867755

rf metrics:
Accuracy: 0.7503571428571428
Precision: 0.31216931216931215
Recall: 0.09394904458598727
F1 score: 0.14443084455324356
ROC AUC: 0.5388331221921151

dt metrics:
Accuracy: 0.6853571428571429
Precision: 0.2761061946902655
Recall: 0.2484076433121019
F1 score: 0.26152556580050296
ROC AUC: 0.529865485448851

xgb metrics:
Accuracy: 0.7671428571428571
Precision: 0.38
Recall: 0.06050955414012739
F1 score: 0.1043956043956044
ROC AUC: 0.5610997231704027
```

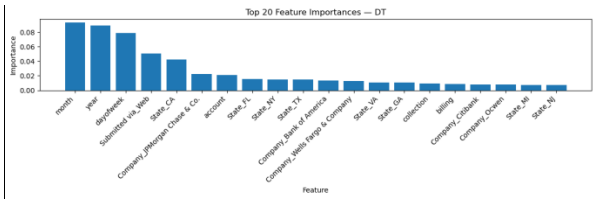
[redacted]

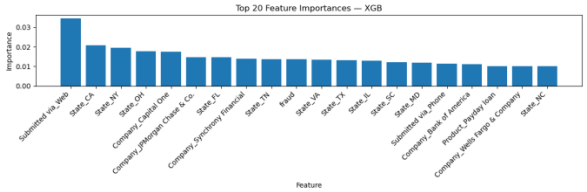
[redacted]



[redacted]







[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]